

Critical Evaluation of Phishing URL Detection Using Machine Learning

Executive Summary

This report reproduces and critically evaluates the public GitHub project "Phishing URL Detection" by Vaibhav Bichave. The source project addresses the cybersecurity problem of identifying phishing websites from URL and website-related indicators. Phishing attacks use deceptive websites and links to steal credentials, financial information, and other sensitive data. Machine learning is a suitable approach because many phishing pages share measurable characteristics such as suspicious URL structure, abnormal redirects, weak domain reputation, misleading anchor links, and unusual traffic signals.

The reproduction uses the public phishing.csv dataset distributed with the source repository. It contains 11,054 samples and 32 raw columns. The target is class. The corrected label convention maps original -1 phishing samples to positive class 1, and original 1 legitimate samples to class 0. This convention makes phishing recall and false negatives directly meaningful: a false negative is a phishing URL incorrectly accepted as legitimate.

The corrected workflow isolates an untouched 20 percent final test set before any model or threshold selection. Logistic Regression, Random Forest, and Gradient Boosting are compared by 5-fold stratified cross-validation on the remaining development data. Random Forest achieves the highest mean development F1 score, 0.9665, and is selected. Its decision threshold is then selected from out-of-fold development probabilities. The rule maximizes phishing recall while requiring at least 0.95 phishing precision, producing a threshold of 0.35.

After the model and threshold are fixed, Random Forest is trained on the full development set and evaluated once on the final test set. It achieves accuracy 0.9738, phishing precision 0.9666, phishing recall 0.9745, F1 0.9705, Matthews Correlation Coefficient 0.9469, and ROC-AUC 0.9966. It misses 25 phishing URLs and incorrectly blocks 33 legitimate URLs. These results support the source's general conclusion that machine learning performs strongly on this benchmark, while the corrected evaluation provides stronger evidence through cross-validation, explicit threshold selection, data-quality checks, feature importance, and a genuinely untouched final test set.

Summary of the Selected Source

The selected source is the GitHub repository "Phishing URL Detection" by Vaibhav Bichave. The repository includes a Jupyter notebook, Python and Flask application files, serialized model artifacts, feature-extraction code, a requirements file, and the phishing.csv dataset. Providing both code and data makes the source more reproducible than a project that describes only an algorithm or reports metrics without implementation files.

The source frames phishing detection as supervised binary classification. Each website is represented by engineered indicators related to its URL, domain, page behavior, and reputation. Candidate models learn patterns that distinguish phishing websites from legitimate ones. The repository reports several classifiers and identifies Gradient Boosting as its best model, with reported accuracy 0.974, F1 0.977, recall 0.994, and precision 0.986. It also reports strong Random Forest and Logistic Regression results.

The source concludes that machine learning can classify phishing URLs effectively and highlights features such as HTTPS, AnchorURL, and WebsiteTraffic as important. This is plausible on the supplied dataset, but the source does not fully document the exact split, random seed, cross-validation procedure, threshold-selection policy, or dataset collection period. The reproduction therefore tests the claim under a more transparent and conservative evaluation design.

Cybersecurity Problem and Motivation

Phishing is a social-engineering attack in which a malicious actor imitates a trusted organization or service. The victim is directed to a deceptive URL and may be asked to enter credentials, payment information, account identifiers, or other confidential data. A phishing page can also deliver malware or redirect users to additional malicious infrastructure.

Automatic URL classification is valuable because blocklists alone cannot identify every newly created phishing site. A machine-learning model can use combinations of weak signals that may not be decisive individually. Examples include an IP address in the URL, excessive subdomains, abnormal redirection, a recently registered domain, suspicious anchor behavior, or inconsistent HTTPS signals.

The costs of classification errors are asymmetric. A missed phishing URL can expose a user to credential theft or malware, whereas a legitimate URL blocked as phishing usually causes a usability or business interruption. Both errors matter, but the security cost of a false negative is generally higher. For that reason, this project reports phishing precision, phishing recall, F1, ROC-AUC, MCC, and the two operational error counts rather than relying on accuracy alone.

Critical Evaluation of the Source

The source has several strengths. It addresses an important and realistic cybersecurity problem, provides executable code and data, compares multiple machine-learning models, and reports high predictive performance. The availability of a notebook and dataset makes an independent reproduction possible. The reported ranking of important features also has a reasonable cybersecurity interpretation.

However, the original evaluation is not documented deeply enough. It is unclear whether the reported metrics come from one split, repeated splits, or cross-validation. The exact random seed and model-selection process are not sufficiently visible. Without these details, it is difficult to determine whether the reported differences between models are stable or specific to one run.

The dataset documentation is also limited. The repository does not clearly explain when the URLs were collected, how they were labeled, whether near-duplicate sites exist, or how representative they are of current phishing campaigns. Phishing tactics change over time, so strong performance on a fixed benchmark does not prove that the same model will perform equally well on future attacks.

Finally, selecting a model or threshold by inspecting the test set makes the reported test estimate optimistic. The corrected reproduction prevents this by separating the final test data first, selecting the model by development cross-validation, and selecting the threshold from out-of-fold development predictions. The final test labels are used only once after all decisions are fixed.

Dataset and Data-Quality Analysis

The dataset contains 11,054 rows and 32 raw columns. Thirty columns are used as predictive features. The Index column is excluded because it is a row identifier rather than a cybersecurity characteristic, and class is the target. All modeling features are numeric and mostly use ordinal values such as -1, 0, and 1.

The original class distribution contains 6,157 legitimate samples and 4,897 phishing samples. After mapping phishing to positive class 1, the phishing rate is approximately 44.3 percent. The dataset is moderately balanced, so accuracy is informative but remains insufficient by itself.

The corrected notebook performs explicit quality checks before modeling:

- Missing values: 0
- Duplicated rows: 0
- Duplicated columns: 0
- Single-value columns: 0
- Modeling features after preprocessing: 30

These checks reduce the risk of obvious data-quality problems. They do not rule out semantic duplicates or multiple samples derived from related domains, because the dataset contains engineered indicators rather than the original raw URLs. This remains a limitation of the available data.

Exploratory Data Analysis

Spearman correlation is used because most features are discrete or ordinal rather than continuous and normally distributed. Spearman measures monotonic rank association and is therefore more appropriate than Pearson correlation for this encoding. Kendall correlation could also be used, but Spearman is computationally efficient and suitable for the dataset size.

The strongest absolute Spearman associations with the phishing label are HTTPS at 0.7358 and AnchorURL at 0.7012. Other notable associations include WebsiteTraffic at 0.3650, PrefixSuffix- at 0.3486, SubDomains at 0.3046, RequestURL at 0.2535, LinksInScriptTags at 0.2509, DomainRegLen at 0.2259, ServerFormHandler at 0.2192, and GoogleIndex at 0.1290.

These values are associations rather than causal effects. A high correlation does not mean that changing one feature would directly cause a site to become legitimate or malicious. It indicates that the encoded feature is useful for distinguishing classes in this benchmark. Correlated predictors may also share information, so individual coefficients and importances must be interpreted cautiously.

Feature Engineering and Preprocessing

The supplied dataset already consists of engineered cybersecurity indicators. The main preprocessing tasks are therefore label definition, identifier removal, feature-target separation, and model-specific scaling.

The original labels are remapped so phishing is positive class 1 and legitimate is class 0. This avoids the confusing convention in the first report, where legitimate URLs were treated as the positive class and security errors had to be interpreted in reverse. The Index column is removed because a row identifier should not be used as a predictive feature.

Logistic Regression is implemented in a scikit-learn Pipeline with StandardScaler. Scaling is appropriate for a linear model because differences in feature scale can affect optimization and coefficient magnitude. Random Forest and Gradient Boosting do not require standardization because tree-based methods split observations by feature thresholds.

No one-hot encoding is needed because all features are numeric. No imputation is needed because the dataset contains no missing values. If missing values were introduced in a future dataset, imputation should be fitted only on development or training data within a Pipeline to prevent leakage.

Additional raw-URL features could include character length, digit count, special-character count, entropy, token patterns, brand similarity, certificate age, WHOIS domain age, and DNS consistency. These could improve modern detection, but they would also introduce dependencies on external services and collection time.

Reproducibility Analysis

The source is partly reproducible because it provides code, data, and dependency information. The revised repository improves reproducibility by exposing the notebook, dataset, README, requirements, report, figures, and machine-readable CSV results directly instead of hiding the implementation inside a ZIP file.

The corrected notebook fixes `RANDOM_STATE = 42`, uses a stratified split, and applies the same 5-fold StratifiedKFold object to candidate-model evaluation and out-of-fold probability generation. The data-loading cell checks the repository data directory, the current directory, and `/content/phishing.csv`, making the notebook usable both locally and in Google Colab.

The experiment can still vary slightly across scikit-learn versions because implementation details and parallel tree construction can change. The repository therefore includes `requirements.txt`, and the notebook records its executed outputs. For stronger long-term reproducibility, exact versions could be pinned in a lock file or environment specification.

Corrected Experimental Design

The experiment begins with a single stratified split: 80 percent development data and 20 percent final test data. This produces 8,843 development rows and 2,211 final test rows. Their phishing rates remain nearly identical at approximately 44.3 percent.

Three candidate models are evaluated:

- Logistic Regression with StandardScaler and a maximum of 1,000 iterations
- Random Forest with 300 trees, balanced class weights, and random state 42
- Gradient Boosting with random state 42

Model selection uses 5-fold stratified cross-validation only on the development set. The primary selection metric is mean phishing F1, while accuracy, precision, recall, ROC-AUC, and standard deviations are also reported. Random Forest obtains the strongest mean F1 and is selected before the final test data is inspected.

After model selection, `cross_val_predict` generates one out-of-fold phishing probability for every development row. Candidate thresholds from 0.20 to 0.80 are evaluated. Among thresholds with phishing precision of at least 0.95, the procedure selects the threshold with the highest phishing recall. This results in threshold 0.35. The model is then refitted on all development rows and evaluated once on the untouched final test set.

Development Cross-Validation Results

model	accuracy mean	accuracy std	precision mean	recall mean	F1 mean	F1 std	ROC-AUC mean
Random Forest	0.9705	0.0010	0.9734	0.9597	0.9665	0.0013	0.9956
Gradient Boosting	0.9485	0.0050	0.9509	0.9321	0.9414	0.0056	0.9898
Logistic Regression	0.9267	0.0059	0.9283	0.9045	0.9162	0.0070	0.9787

Random Forest leads across the main development metrics and shows low fold-to-fold variation. Gradient Boosting, although reported as the best model by the source, is second in this reproduction. Logistic Regression remains a useful baseline but cannot capture nonlinear interactions as effectively as the tree ensembles.

The cross-validation results are close to the earlier single-split estimates, which supports the stability of the model ranking. More importantly, the selection is now based on repeated held-out development folds instead of the final test set.

Threshold Tuning

The default threshold of 0.50 is not automatically optimal for a security application. Lowering the phishing threshold generally increases recall and reduces missed phishing URLs, while increasing the number of legitimate URLs blocked. Raising it has the opposite effect.

The selected threshold of 0.35 is the lowest tested threshold whose out-of-fold phishing precision exceeds 0.95. On the development predictions it produces precision 0.9538, recall 0.9745, and F1 0.9640. At threshold 0.50, precision rises to 0.9733 but recall falls to 0.9597. The selected threshold therefore reflects the stated security priority of reducing missed phishing while maintaining a strong precision floor.

The out-of-fold counts in threshold tuning cover the full 8,843-row development set and must not be compared directly with the smaller final-test counts. Their purpose is threshold selection, not final performance reporting.

Final Untouched Test Results

model	threshold	accuracy	phishing precision	phishing recall	F1	MCC	ROC-AUC
Random Forest	0.35	0.9738	0.9666	0.9745	0.9705	0.9469	0.9966

The final confusion matrix contains:

- True positives: 954 phishing URLs detected correctly
- True negatives: 1,199 legitimate URLs accepted correctly
- False negatives: 25 phishing URLs missed
- False positives: 33 legitimate URLs blocked incorrectly

The final accuracy of 0.9738 indicates that 97.38 percent of the final test samples are classified correctly. Phishing precision of 0.9666 means that most URLs flagged as phishing are truly phishing. Phishing recall of 0.9745 means that the system detects approximately 97.45 percent of phishing samples. F1 of 0.9705 summarizes the balance between these two quantities.

MCC of 0.9469 indicates strong balanced classification across all four confusion-matrix cells. ROC-AUC of 0.9966 shows excellent ranking performance across possible thresholds, but ROC-AUC alone does not determine the operational threshold or capture the asymmetric costs of errors.

Comparison with the Original Source

The source reports Gradient Boosting accuracy of 0.974 and Random Forest accuracy of 0.967. The corrected reproduction obtains Random Forest final-test accuracy of 0.9738, which is close to both reported values. Exact equality is not expected because the source does not fully document its split, preprocessing, threshold, or package versions.

The important difference is methodological. The corrected result is produced after model selection by development cross-validation and threshold selection by out-of-fold development predictions. The final test set is not reused for these decisions. This makes the reported final result a more defensible estimate of performance on unseen samples from the same benchmark distribution.

Error Analysis

The 25 false negatives are the most security-critical errors because they are phishing URLs predicted as legitimate. Even a small false-negative rate can matter in deployment, where a missed link may lead to credential theft, malware installation, or financial loss. The selected threshold reduces these errors relative to a more conservative phishing threshold.

The 33 false positives are legitimate URLs predicted as phishing. These errors reduce usability, may block business activity, and can cause users to ignore future warnings. The selected precision constraint prevents recall optimization from creating an excessive false-alarm rate.

The equal cost assumption used by standard accuracy is therefore inappropriate as the only decision criterion. A production system should define explicit costs, monitor both error types, and possibly use different policies for warning, blocking, and manual review. Thresholds may also need to vary by user role, asset sensitivity, or threat environment.

Feature Importance

feature	importance
HTTPS	0.3264
AnchorURL	0.2421
WebsiteTraffic	0.0734
SubDomains	0.0639
PrefixSuffix-	0.0439
LinksInScriptTags	0.0421
ServerFormHandler	0.0223
RequestURL	0.0194
LinksPointingToPage	0.0185
DomainRegLen	0.0173
AgeofDomain	0.0158
UsingIP	0.0126

HTTPS and AnchorURL dominate the Random Forest impurity-based importance ranking. WebsiteTraffic, SubDomains, PrefixSuffix-, and LinksInScriptTags also contribute meaningful predictive information. The ranking broadly agrees with the correlation analysis and the source's discussion.

Impurity-based importance can favor features with more possible split points and can distribute importance unpredictably among correlated predictors. These values describe how the fitted model uses the benchmark features; they are not causal explanations. Permutation importance or SHAP analysis on a separate interpretation set would provide additional evidence.

Limitations

The largest limitation is dataset age and provenance. The collection period and labeling process are not documented sufficiently, and phishing behavior changes rapidly. Random splitting assumes that development and test rows come from the same distribution. It does not measure performance on future phishing campaigns.

The dataset contains engineered feature values rather than raw URLs. This prevents direct inspection of semantic duplicates, domain families, and temporal relationships. It also means the experiment does not evaluate the reliability, latency, or cost of extracting these indicators in a live system.

Model selection and threshold tuning are now separated from the final test, but the same benchmark still guides the overall research design. External validation on a newer dataset would be necessary before claiming broad real-world effectiveness. Adversaries may also adapt specifically to evade features that detectors rely on.

Conclusions

The corrected reproduction supports the source's general claim that machine learning can classify phishing URLs accurately on the supplied benchmark. Random Forest is the strongest model in development cross-validation and achieves final-test accuracy 0.9738, phishing recall 0.9745, phishing precision 0.9666, F1 0.9705, MCC 0.9469, and ROC-AUC 0.9966 at threshold 0.35.

The project also demonstrates why cybersecurity evaluation requires more than accuracy. Missed phishing and blocked legitimate sites have different consequences. Cross-validation, out-of-fold threshold tuning, explicit label semantics, confusion-matrix counts, MCC, ROC-AUC, and feature analysis provide a more complete and reproducible evaluation.

The result should be interpreted as strong performance on one fixed dataset, not proof of a complete production defense. A deployable system would require fresh data, time-based validation, external testing, robust feature extraction, monitoring, retraining, and operational cost analysis.

Future Improvements

Future work should document the dataset collection date, source of URLs, labeling procedure, and filtering rules. Raw URLs or stable sample identifiers would enable duplicate-family analysis and group-aware splitting. Timestamps would enable temporal validation in which the model is trained on older attacks and tested on newer ones.

The model comparison could be extended to calibrated boosting methods, XGBoost, LightGBM, and cost-sensitive learning. Hyperparameter tuning should be nested within development cross-validation so that the final test remains untouched. Precision-recall curves and expected-cost analysis could support more principled threshold decisions.

Interpretability could be strengthened with permutation importance, SHAP values, partial-dependence analysis, and error inspection grouped by feature patterns. Deployment research should examine data drift, adversarial manipulation, latency, missing external signals, and human interaction with warnings.

Finally, dependencies should be pinned exactly, automated tests should verify data schema and metric generation, and continuous integration should execute the notebook or an equivalent script. These steps would make future reproductions easier and reduce the chance that results become stale.

References

- Vaibhav Bichave, "Phishing URL Detection", GitHub repository: <https://github.com/vaibhavbichave/Phishing-URL-Detection>
- Dataset file phishing.csv: <https://github.com/vaibhavbichave/Phishing-URL-Detection/blob/master/phishing.csv>
- Scikit-learn model evaluation documentation: https://scikit-learn.org/stable/modules/model_evaluation.html
- Reproduction notebook: [notebooks/phishing_url_detection_enhanced.ipynb](#)